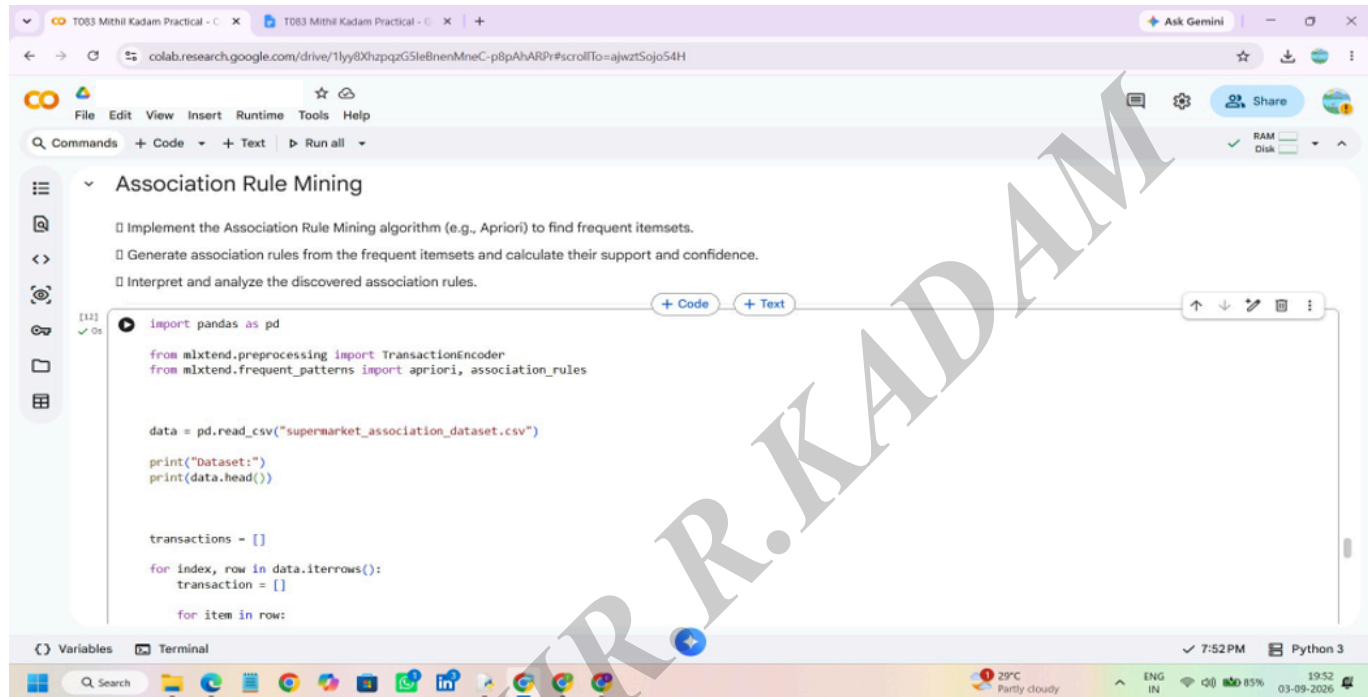


Aim:- Association Rule Mining Implement the Association Rule Mining algorithm (e.g., Apriori) to find frequent itemsets.

Generate association rules from the frequent itemsets and calculate their support and confidence. Interpret and analyze the discovered association rules.



The screenshot shows a Google Colab notebook titled "Association Rule Mining". The code in the first cell is as follows:

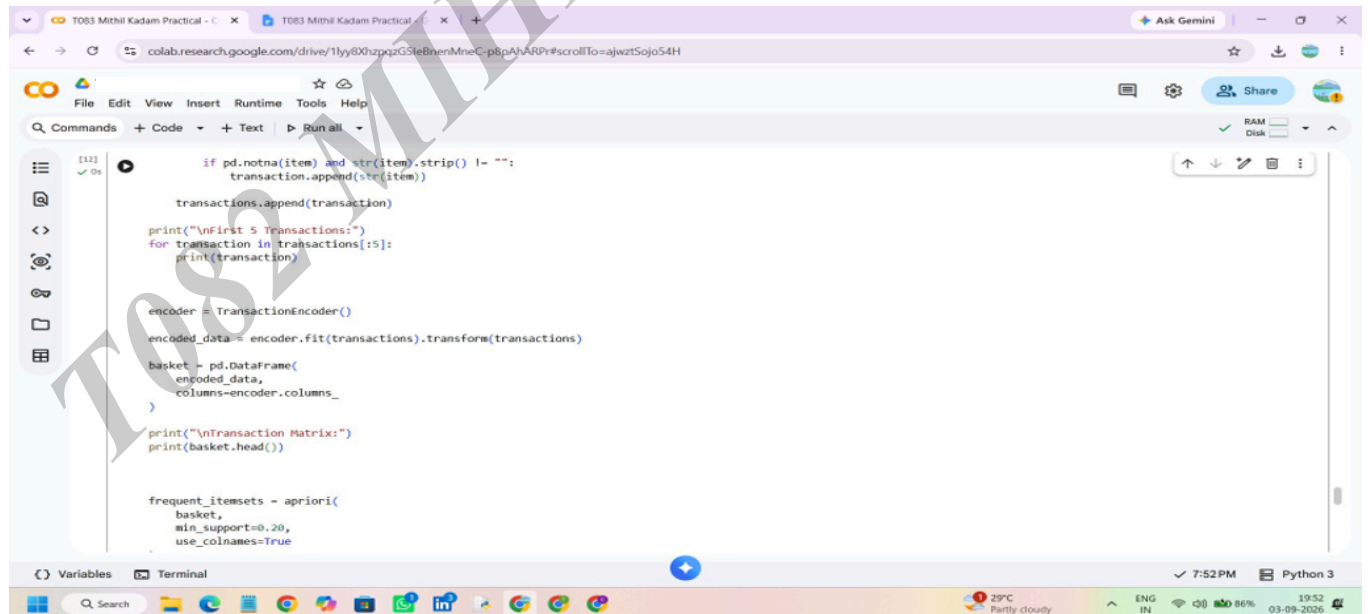
```
import pandas as pd

from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules

data = pd.read_csv("supermarket_association_dataset.csv")
print("Dataset:")
print(data.head())

transactions = []

for index, row in data.iterrows():
    transaction = []
    for item in row:
```



The screenshot shows the continuation of the code in the second cell of the Google Colab notebook:

```
        transaction.append(str(item))

    transactions.append(transaction)

print("\nFirst 5 Transactions:")
for transaction in transactions[:5]:
    print(transaction)

encoder = TransactionEncoder()
encoded_data = encoder.fit(transactions).transform(transactions)

basket = pd.DataFrame(
    encoded_data,
    columns=encoder.columns_
)

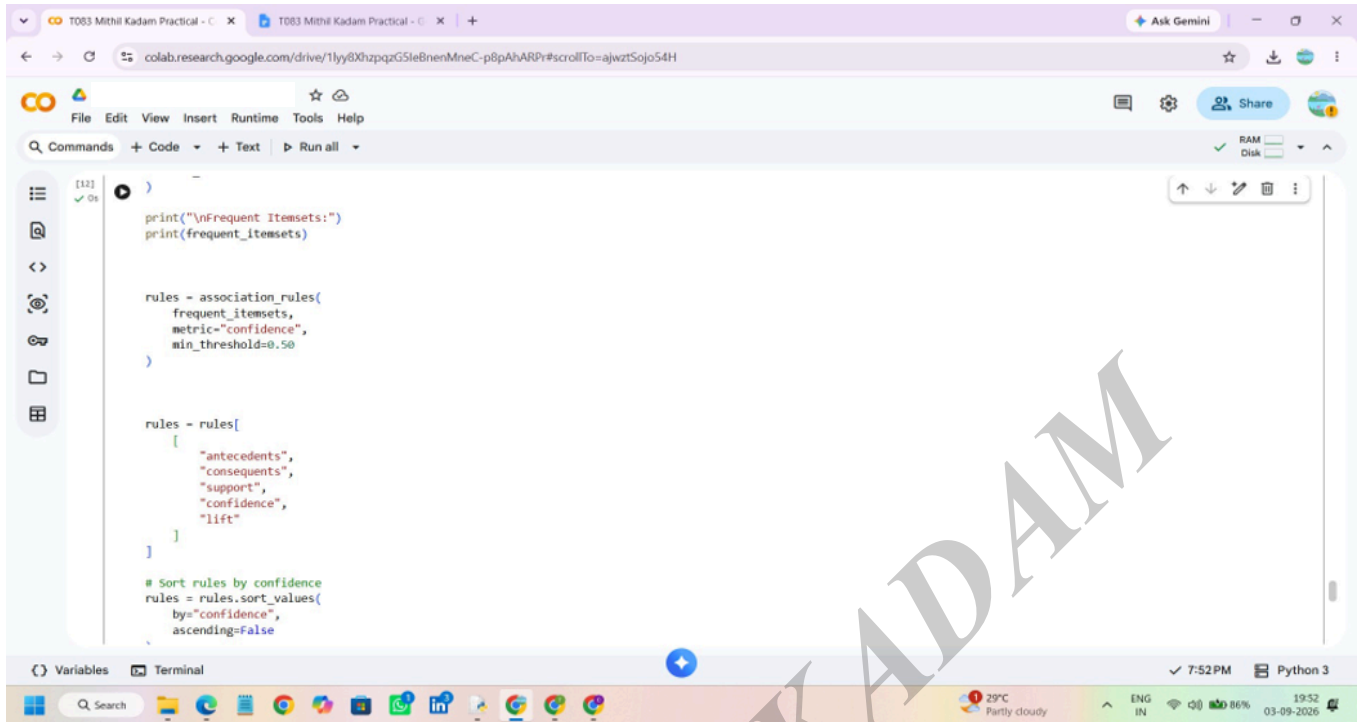
print("\nTransaction Matrix:")
print(basket.head())

frequent_itemsets = apriori(
    basket,
    min_support=0.20,
    use_colnames=True
```

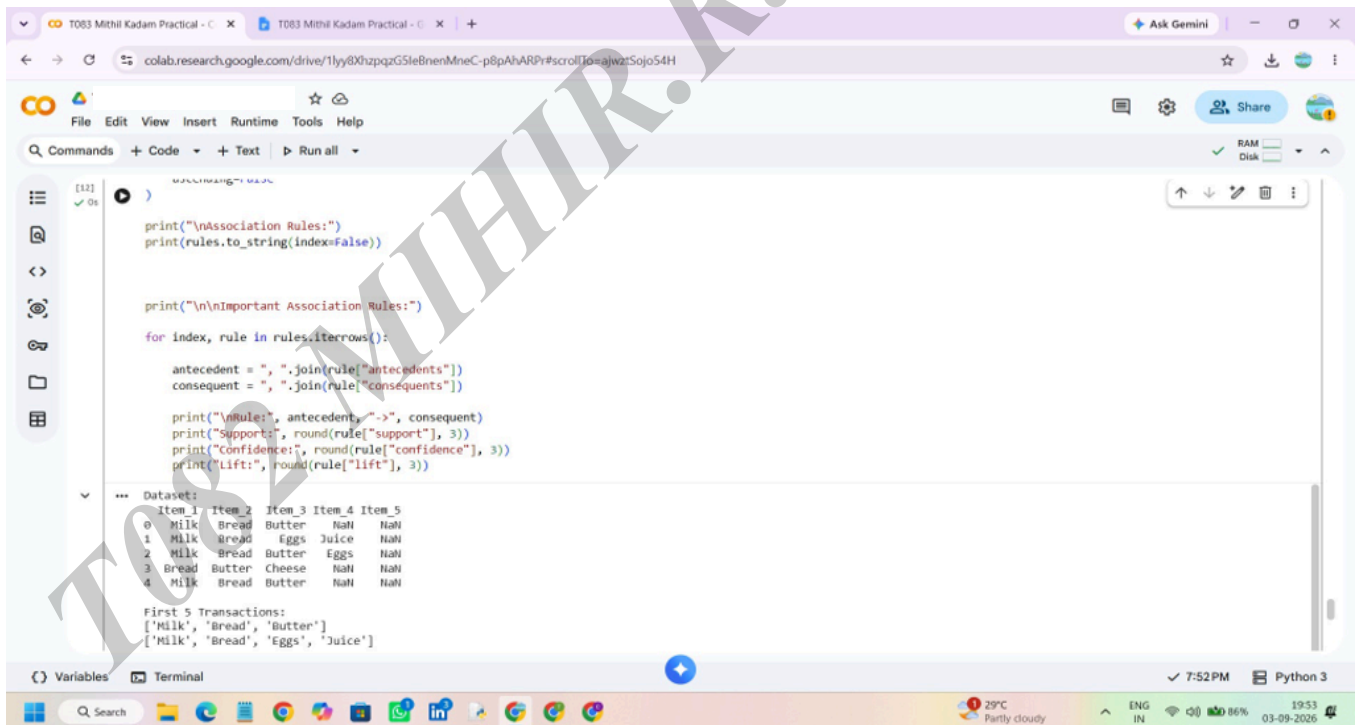
Name:- MIHIR.R.KADAM

Roll No. T082

Subject:- Artificial Intelligence



```
[12] ✓ 0s  
print("\nFrequent Itemsets:")  
print(frequent_itemsets)  
  
rules = association_rules(  
    frequent_itemsets,  
    metric="confidence",  
    min_threshold=0.50  
)  
  
rules = rules[  
    [  
        "antecedents",  
        "consequents",  
        "support",  
        "confidence",  
        "lift"  
    ]  
]  
  
# Sort rules by confidence  
rules = rules.sort_values(  
    by="confidence",  
    ascending=False  
)
```



```
[12] ✓ 0s  
print("\nAssociation Rules:")  
print(rules.to_string(index=False))  
  
print("\n\nImportant Association Rules:")  
for index, rule in rules.iterrows():  
    antecedent = ", ".join(rule["antecedents"])  
    consequent = ", ".join(rule["consequents"])  
  
    print("\nrule:", antecedent, "->", consequent)  
    print("Support:", round(rule["support"], 3))  
    print("Confidence:", round(rule["confidence"], 3))  
    print("Lift:", round(rule["lift"], 3))  
  
Dataset:  
  Item_1 Item_2 Item_3 Item_4 Item_5  
0  Milk  Bread  Butter  NaN  NaN  
1  Milk  Bread  Eggs  Juice  NaN  
2  Milk  Bread  Butter  Eggs  NaN  
3  Bread  Butter  Cheese  NaN  NaN  
4  Milk  Bread  Butter  NaN  NaN  
  
First 5 Transactions:  
['Milk', 'Bread', 'Butter']  
['Milk', 'Bread', 'Eggs', 'Juice']
```

Name:- MIHIR.R.KADAM

Roll No. T082

Subject:- Artificial Intelligence

The screenshot shows a Google Colab notebook with the following content:

```
Dataset:
Item 1 Item 2 Item 3 Item 4 Item 5
0 Milk Bread Butter NaN NaN
1 Milk Bread Eggs Juice NaN
2 Milk Bread Butter Eggs NaN
3 Bread Butter Cheese NaN NaN
4 Milk Bread Butter NaN NaN

First 5 Transactions:
['Milk', 'Bread', 'Butter']
['Milk', 'Bread', 'Eggs', 'Juice']
['Milk', 'Bread', 'Butter', 'Eggs']
['Bread', 'Butter', 'Cheese']
['Milk', 'Bread', 'Butter']

Transaction Matrix:
Apples Bananas Biscuits Bread Butter Cheese Chicken Eggs Juice \
0 False False False True True False False False False
1 False False False True False False False True True
2 False False False True True False False True True
3 False False False True True True False False False
4 False False False True True False False False False

Milk Rice Sugar Tea
0 True False False False
1 True False False False
2 True False False False
3 False False False False
4 True False False False

Frequent Itemsets:
support itemsets
```

The screenshot shows a Google Colab notebook with the following content:

```
(Milk) (Bread) 0.40 0.571429 0.879121
(Bread, Milk) (Butter) 0.20 0.500000 2.000000

Important Association Rules:

Rule: Tea -> Biscuits
Support: 0.2
Confidence: 1.0
Lift: 3.333

Rule: Butter -> Bread
Support: 0.25
Confidence: 1.0
Lift: 1.538

Rule: Butter, Milk -> Bread
Support: 0.2
Confidence: 1.0
Lift: 1.538

Rule: Rice -> Chicken
Support: 0.2
Confidence: 1.0
Lift: 5.0

Rule: Chicken -> Rice
Support: 0.2
Confidence: 1.0
Lift: 5.0

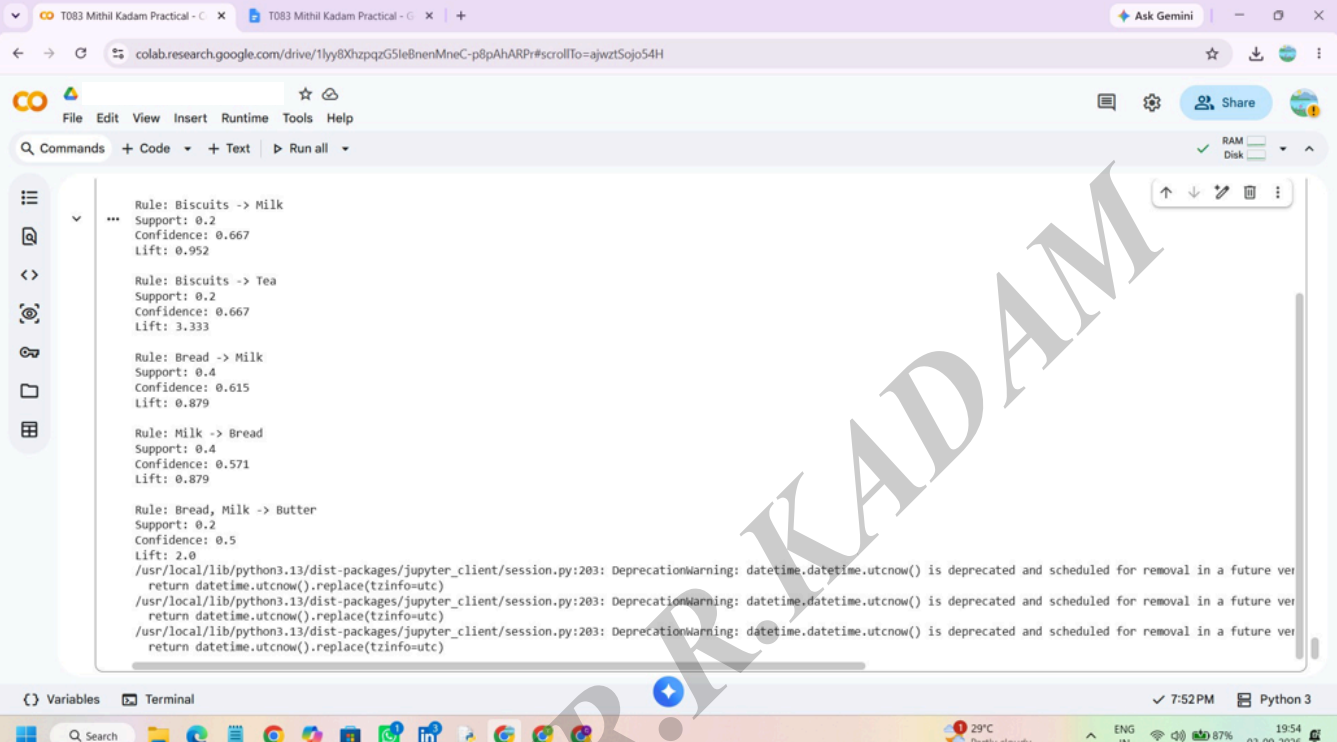
Rule: Butter -> Bread, Milk
```

Name:- MIHIR.R.KADAM

Roll No. T082

Subject:- Artificial Intelligence

Sheth L.U.J. And Sir M.V. College



```
Rule: Biscuits -> Milk
Support: 0.2
Confidence: 0.667
Lift: 0.952

Rule: Biscuits -> Tea
Support: 0.2
Confidence: 0.667
Lift: 3.333

Rule: Bread -> Milk
Support: 0.4
Confidence: 0.615
Lift: 0.879

Rule: Milk -> Bread
Support: 0.4
Confidence: 0.571
Lift: 0.879

Rule: Bread, Milk -> Butter
Support: 0.2
Confidence: 0.5
Lift: 2.0

/usr/local/lib/python3.13/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version; use datetime.datetime.now(datetime.UTC) instead
  return datetime.datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.13/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version; use datetime.datetime.now(datetime.UTC) instead
  return datetime.datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.13/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version; use datetime.datetime.now(datetime.UTC) instead
  return datetime.datetime.utcnow().replace(tzinfo=utc)
```

Name:- MIHIR.R.KADAM

Roll No. T082

Subject:- Artificial Intelligence